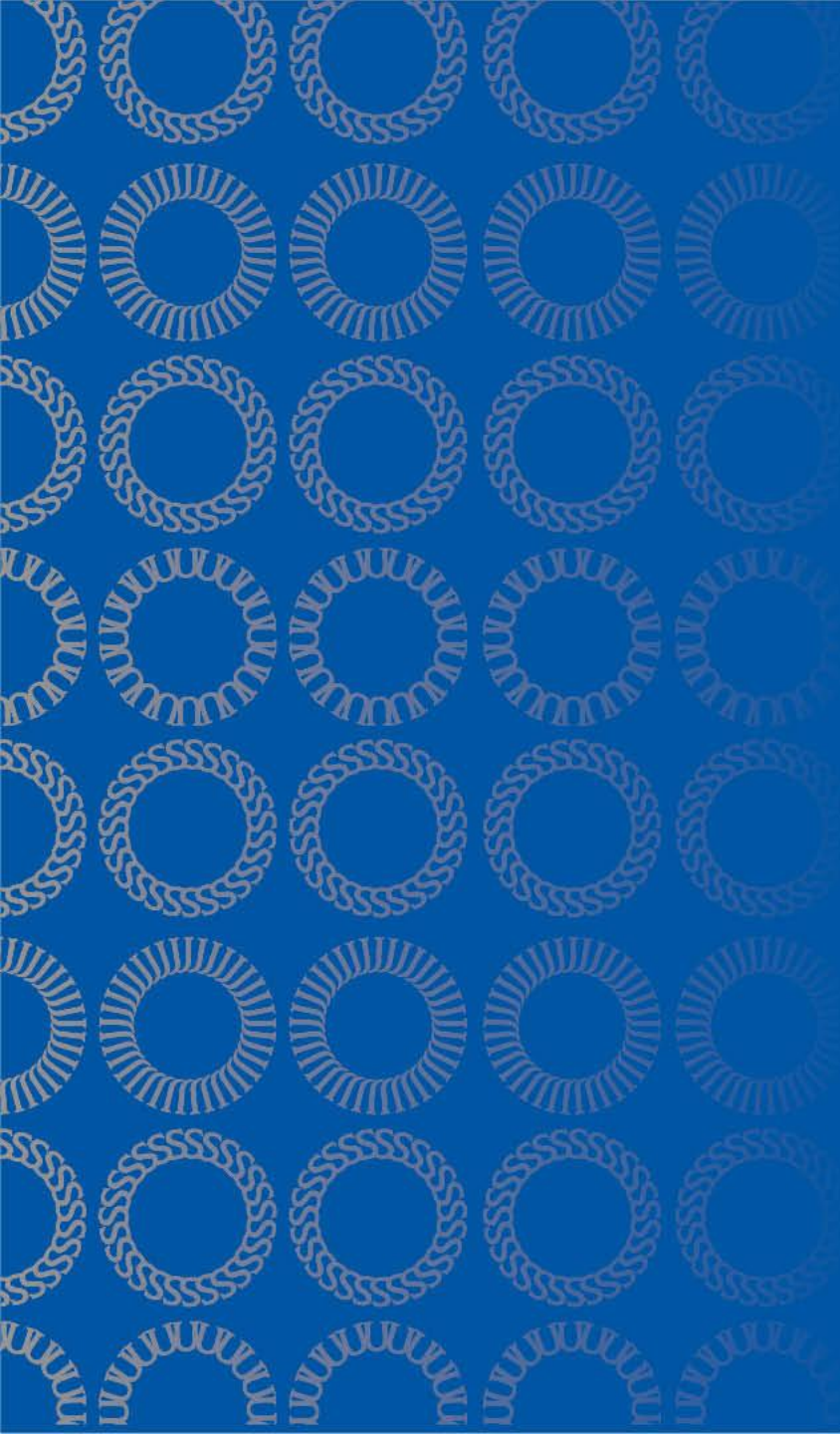




DATA 255

Deep Learning and Computer Vision in Applied Data Intelligence

Aug 26, 2026

Taehee Jeong, Ph.D.

San Jose State University

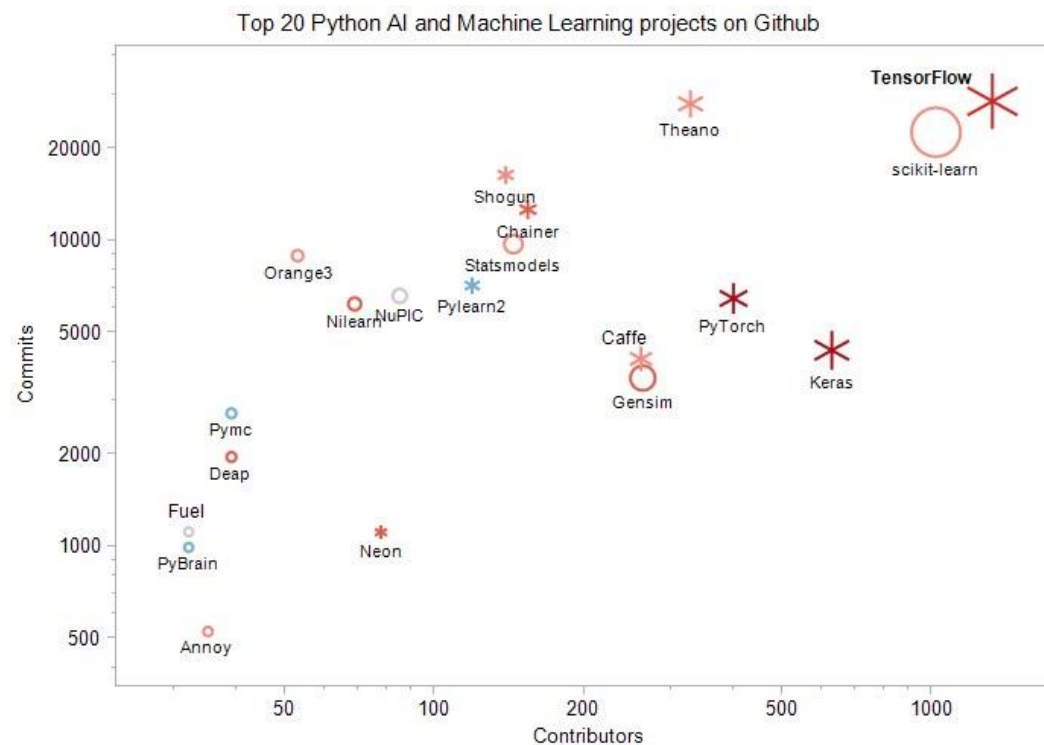
Topics covered in this Course

- Fundamentals of Neural network
- Deep neural network (DNN)
- Convolutional neural networks (CNN)
- Autoencoder and VAE
- Generative adversarial networks (GAN)
- Diffusion models

Today's Topics

- Deep Learning Framework
- Generative AI
- Fundamental Deep neural networks
- Quantization

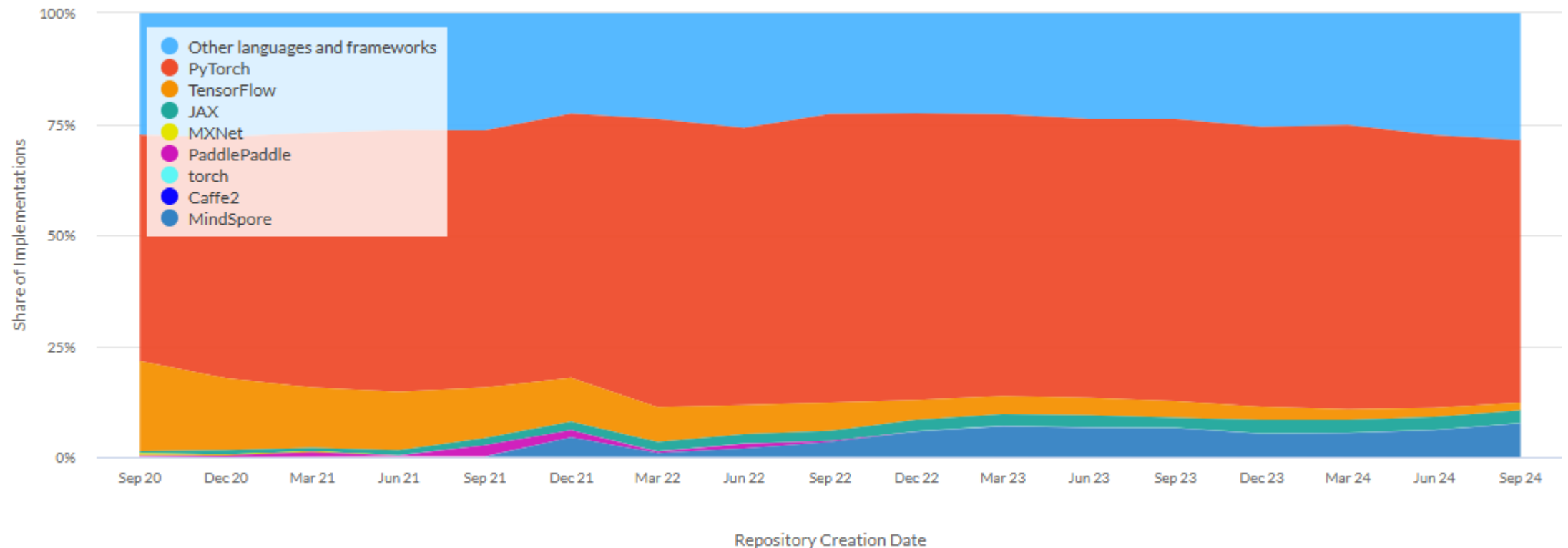
Deep learning Framework



Top 20 Python AI and Machine Learning Open Source Projects (2018)
By Ilan Reinstein, KDnuggets

Deep learning Framework

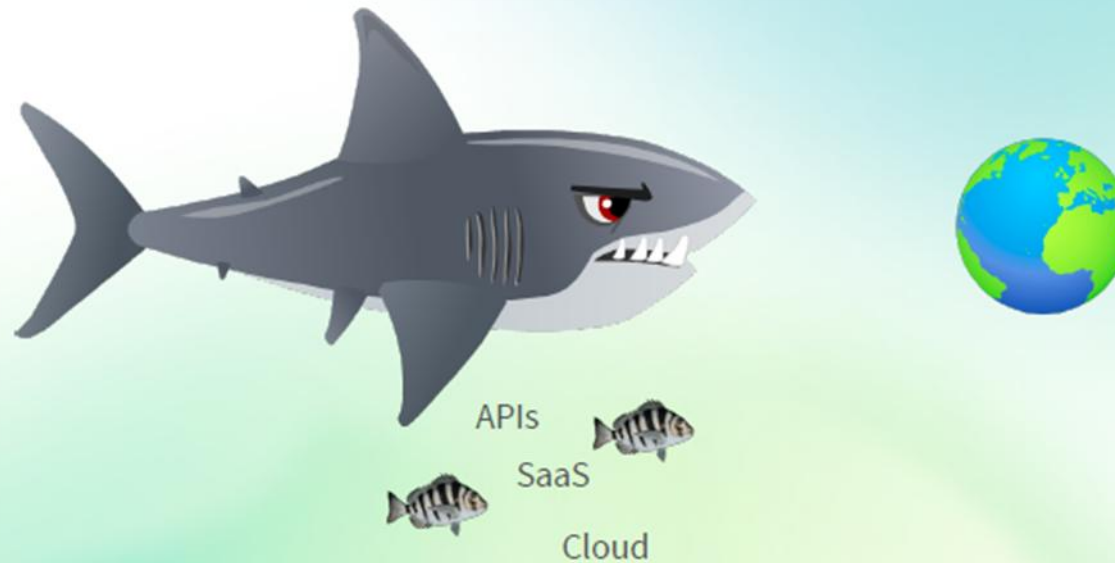
Paper Implementations grouped by framework



Source: <https://paperswithcode.com/trends>

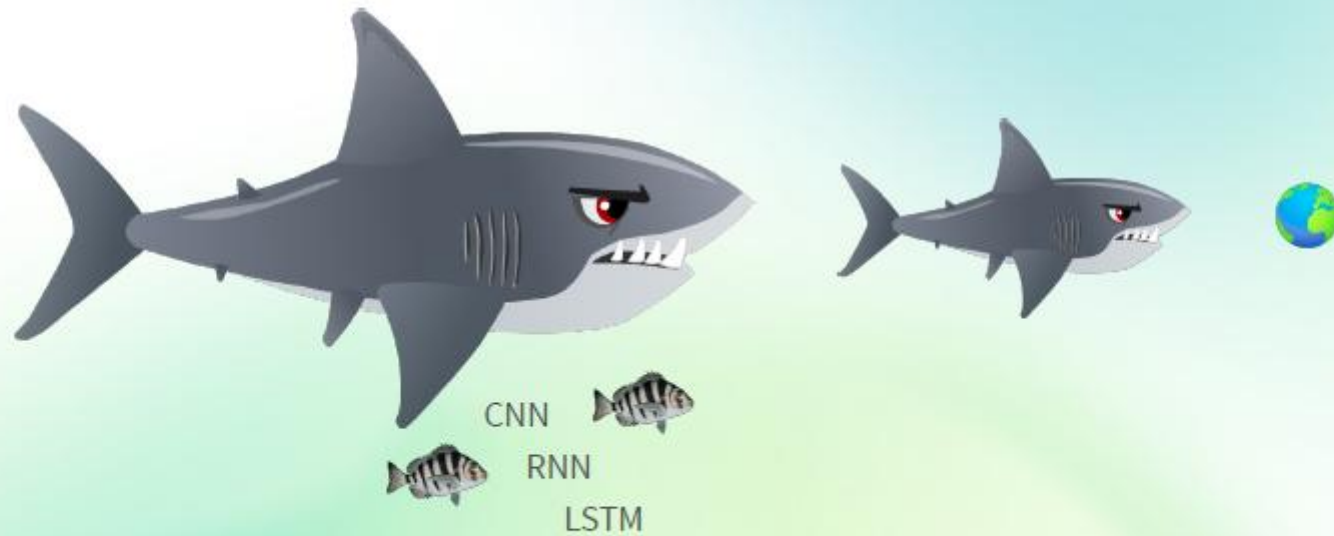
2011: Software is eating the world

<https://a16z.com/2011/08/20/why-software-is-eating-the-world/>



2017: Deep Learning is eating Software

<https://www.technologyreview.com/2017/05/12/151722/nvidia-ceo-software-is-eating-the-world-but-ai-is-going-to-eat-software/>



2022: Transformers are eating Deep Learning

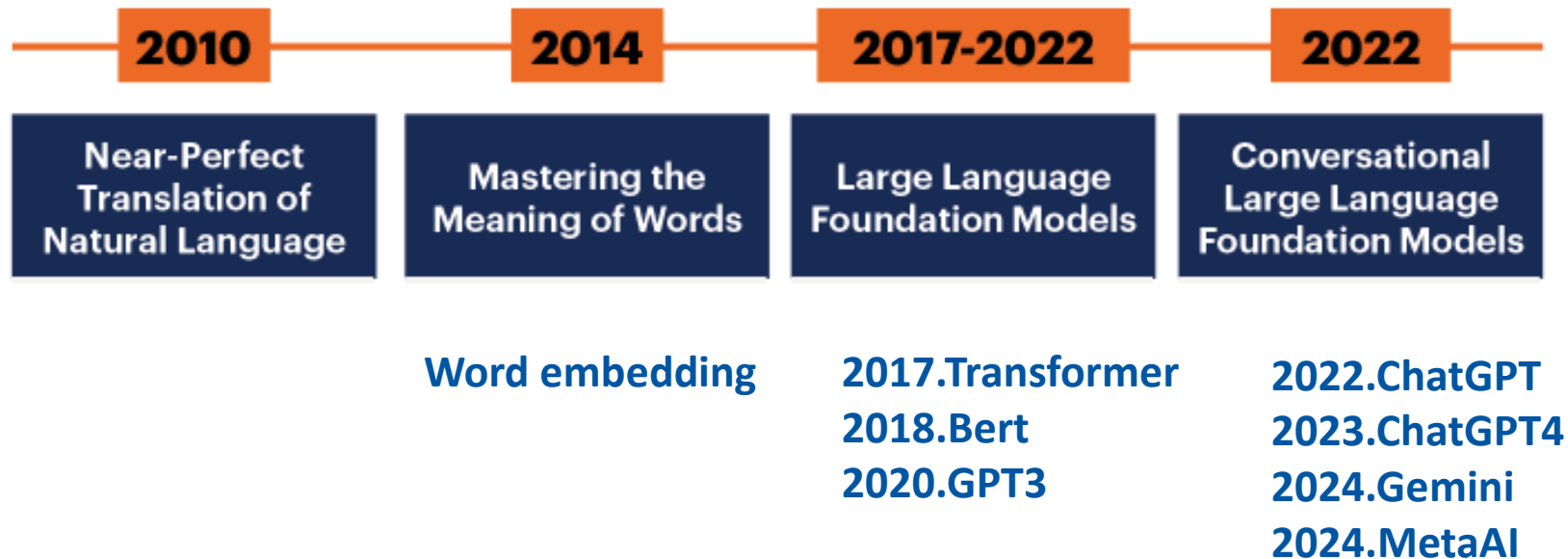


Generative AI

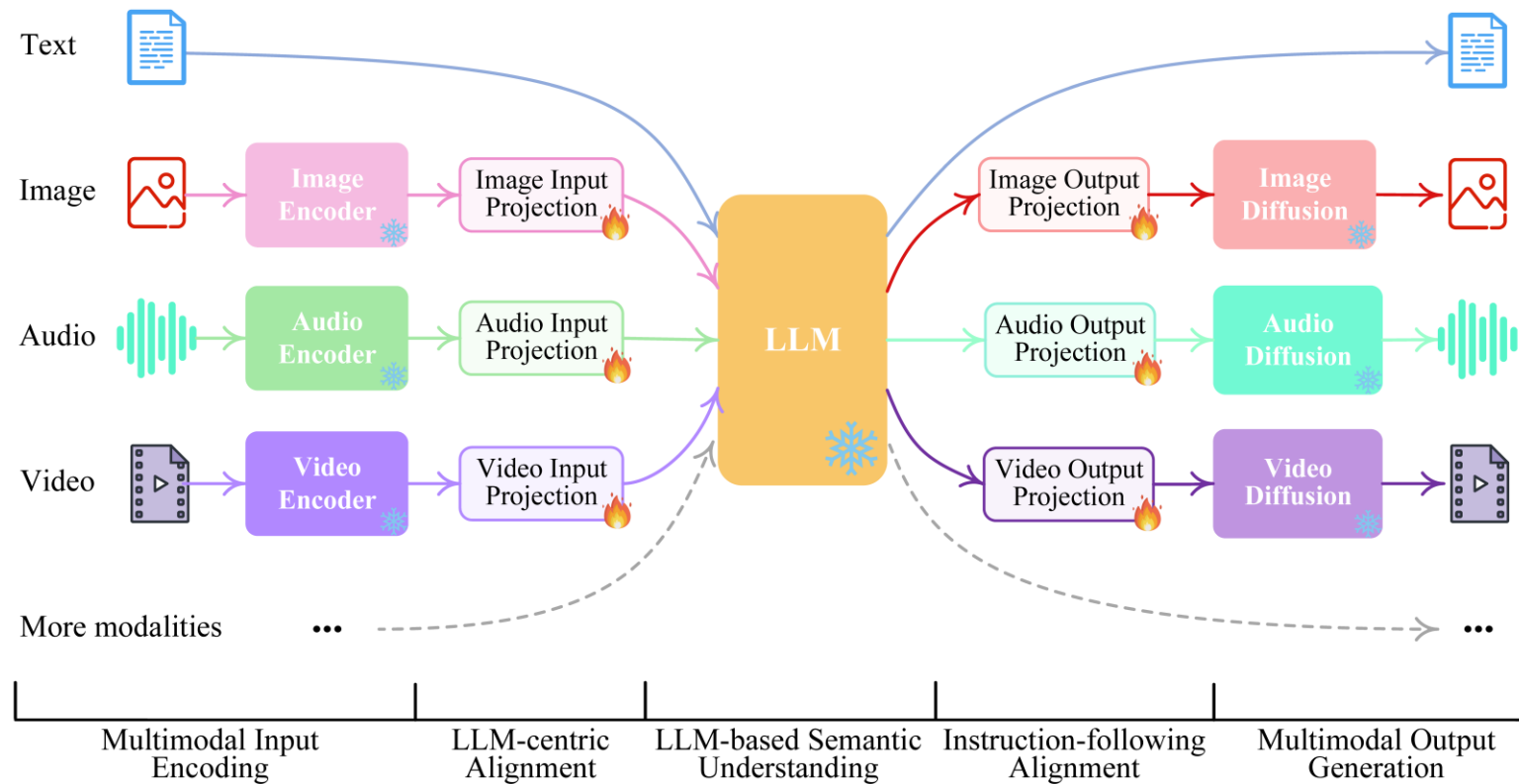
Generative AI can learn from existing artifacts to generate new, realistic artifacts (at scale) that reflect the characteristics of the training data but don't repeat it.

It can produce a variety of content, such as **images, video, music, speech, text, software code and product designs**.

The Journey to Generative AI



Multimodal-LLMs



Source from <https://github.com/NExT-GPT/>

Text-image-generation

(Closed source)

- DALL·E 3
- Imagen 2.5

(Open source)

- Stable Diffusion 3

DALL·E 3: text-to-image synthesis



[illegible]

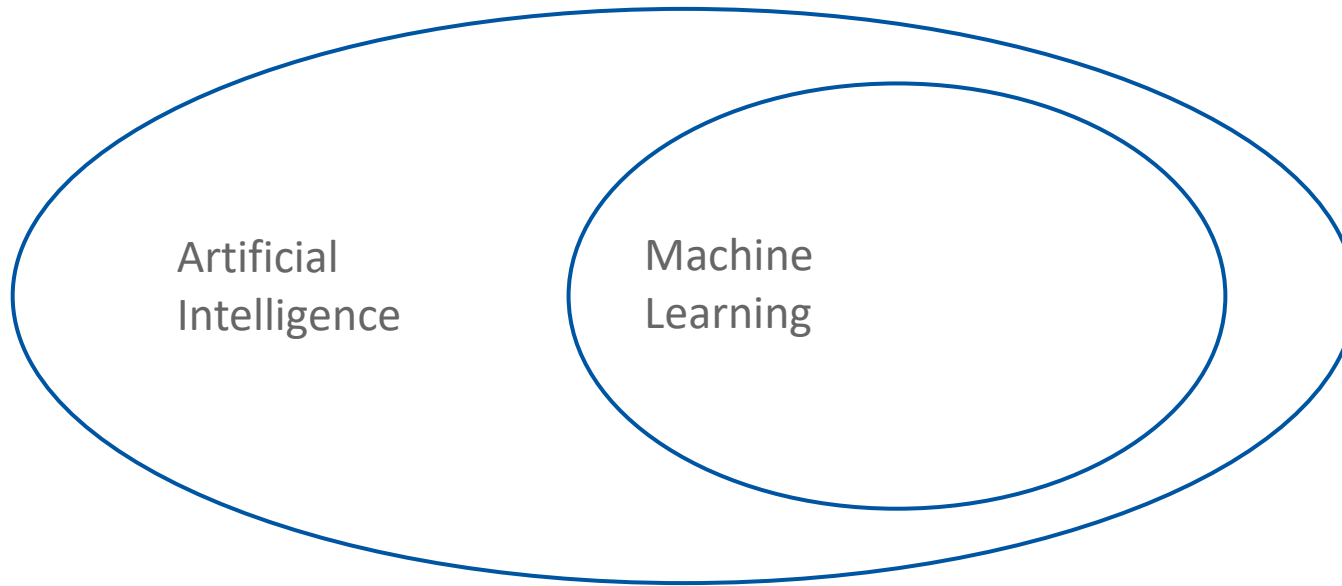
SU SAN JOSÉ STATE
UNIVERSITY

Stable diffusion 3



AI versus ML

“Artificial intelligence (AI)—defined as a system’s ability to correctly interpret external data, to learn from such data, and to use those learnings to achieve specific goals and tasks through flexible adaptation” from Kaplan and Haenlein



AI: Artificial Intelligence

ML: Machine Learning

What Is Machine Learning?

[Machine Learning is the] field of study that gives computers the ability to learn without being explicitly programmed.

—Arthur Samuel, 1959

A computer program is said to learn from experience E with respect to some task T and some performance measure P , if its performance on T , as measured by P , improves with experience E .

—Tom Mitchell, 1997.

What Is Machine Learning?

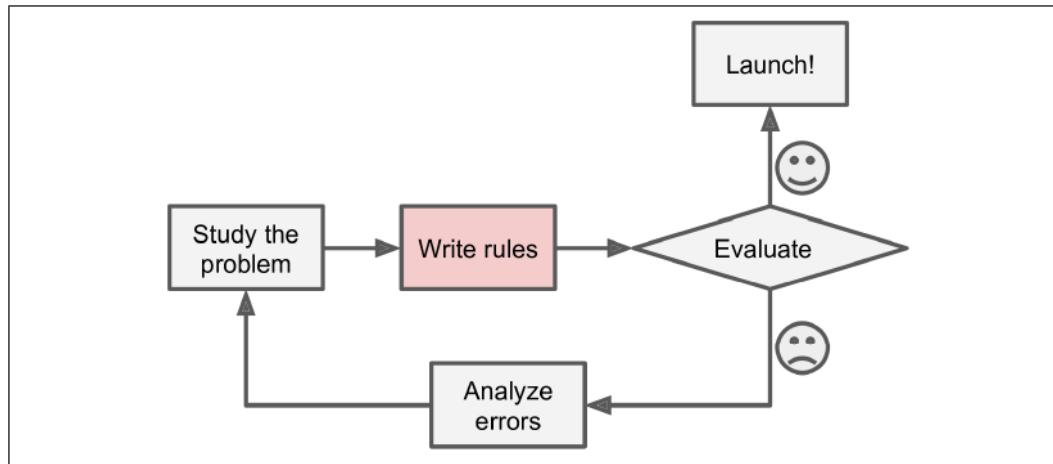


Figure 1-1. The traditional approach

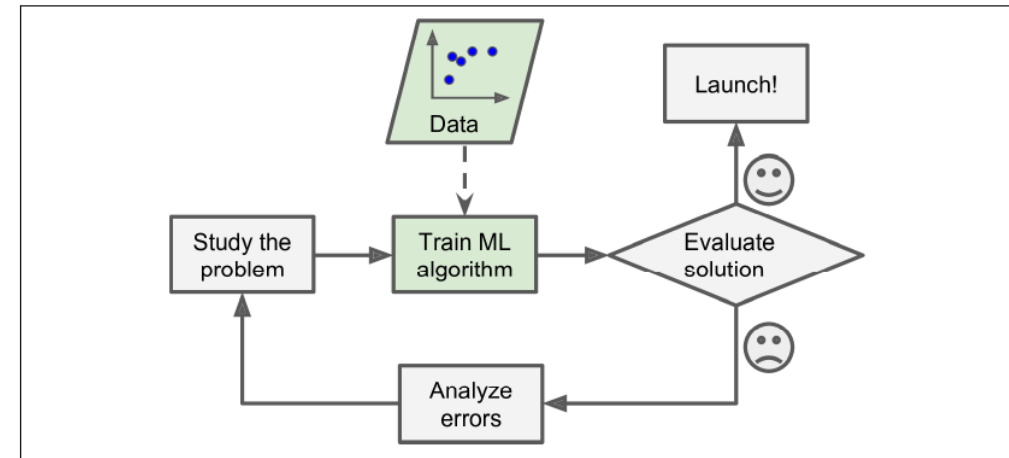
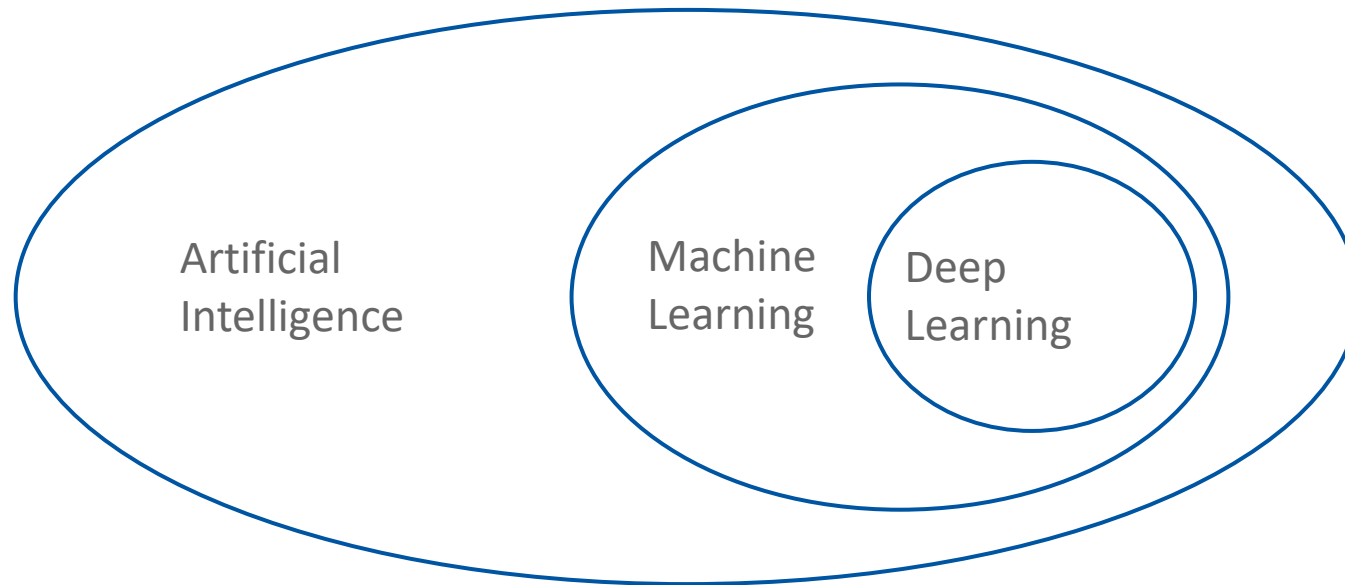


Figure 1-2. Machine Learning approach

Regression vs. Classification in ML

- In a regressor, the model tries to emit a continuous numerical value given a specific input.
- Classifiers have a set of prespecified classes. Given a specific input, they try to emit the class to which the input belongs.

Machine Learning vs. Deep Learning



Biological Neurons

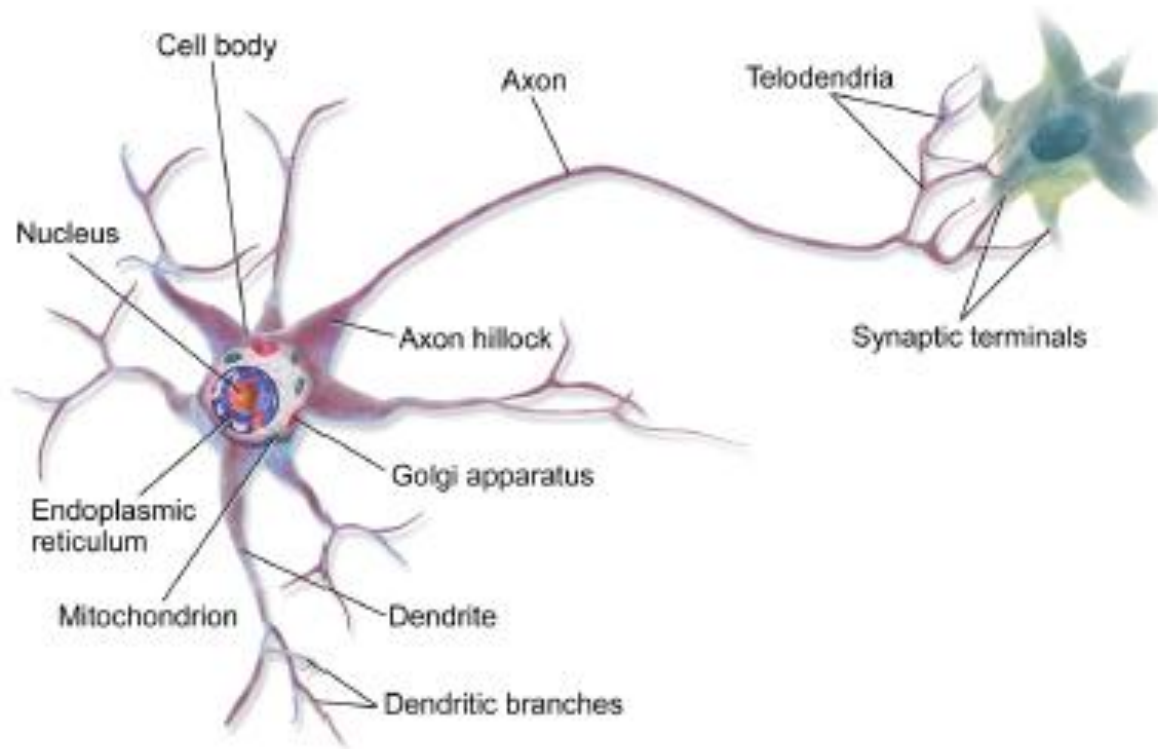
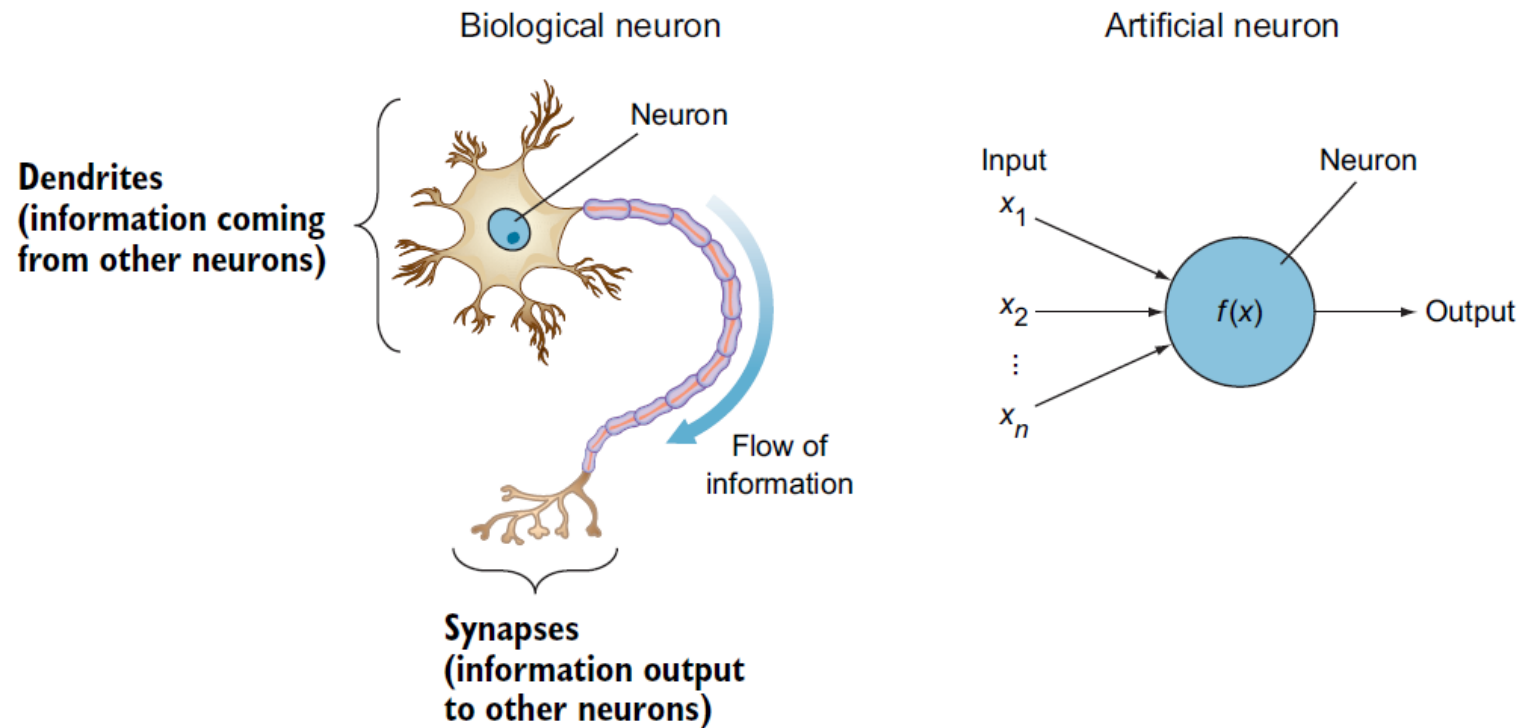
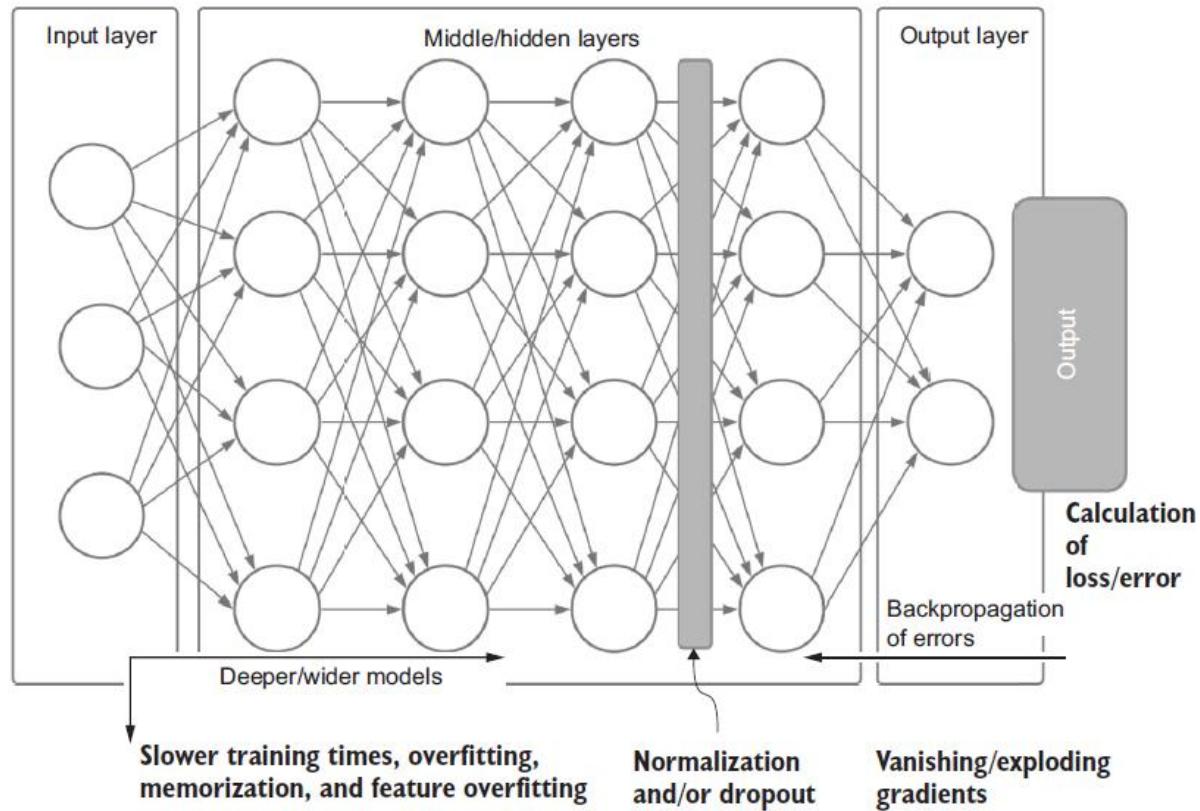


Image by Bruce Blaus ([Creative Commons 3.0](https://en.wikipedia.org/wiki/Neuron)). Reproduced from <https://en.wikipedia.org/wiki/Neuron>.

Similarities between biological neurons and artificial systems



Neural network architecture

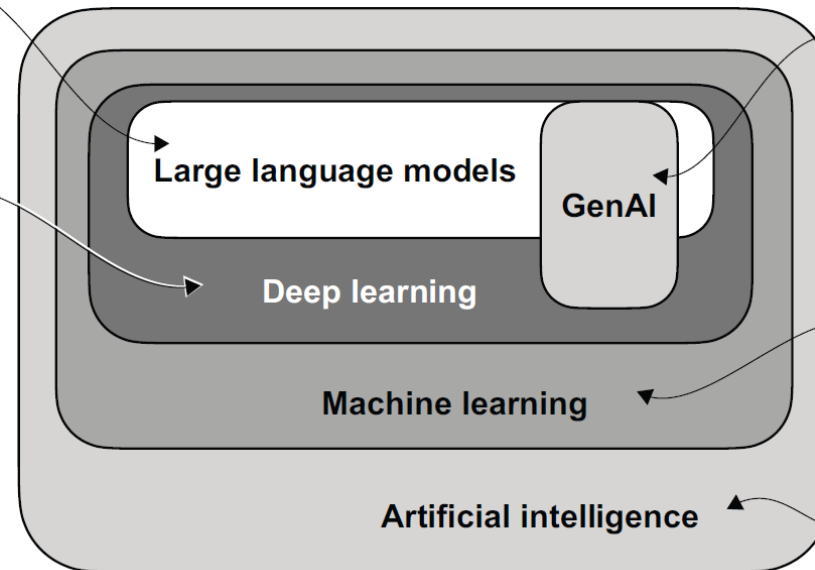


Source: Evolutionary Deep Learning, Micheal Lanham

Generative AI

Deep neural network for parsing and generating human-like text

Machine learning with neural networks consisting of many layers



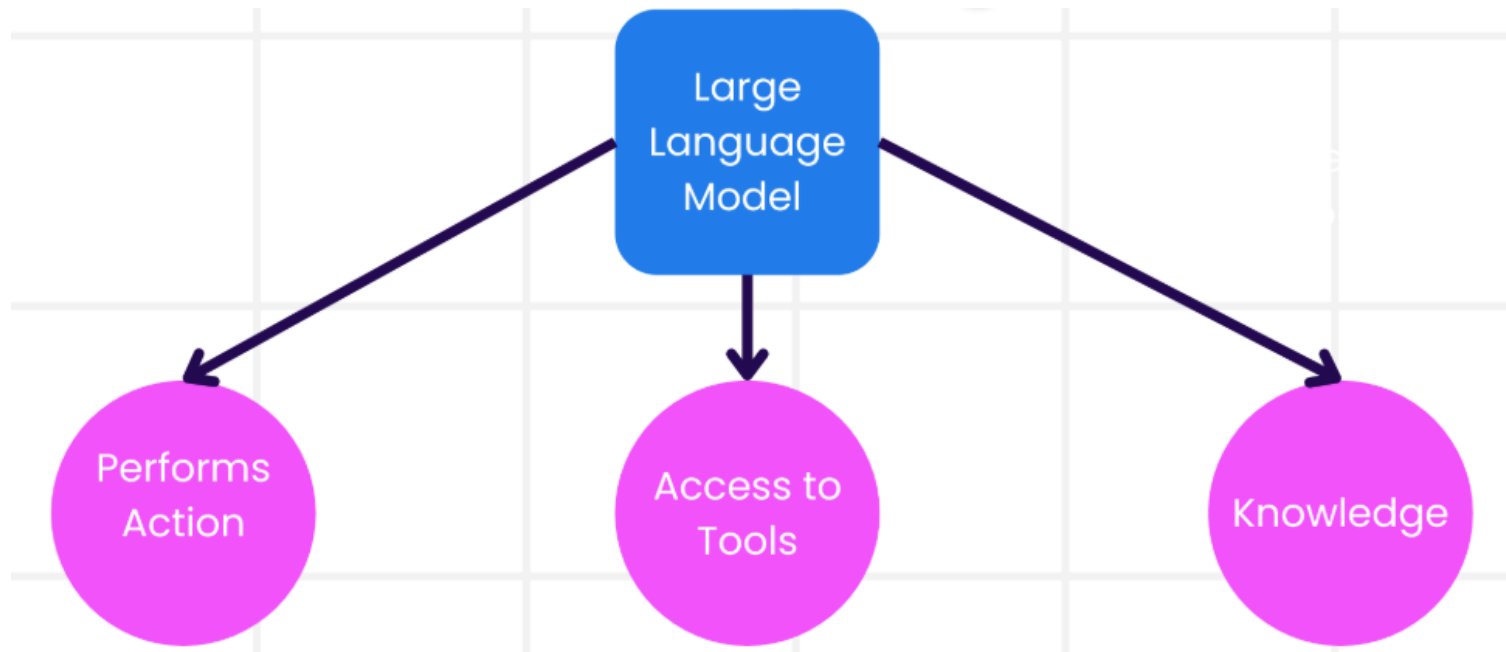
GenAI involves the use of deep neural networks to create new content, such as text, images, or various forms of media

Algorithms that learn rules automatically from data

Systems with human-like intelligence

What are AI Agents?

AI Agents are systems that let Large Language Models (LLMs) take actions — by giving them tools and knowledge to act on the world, not just respond to prompts.



Source: <https://github.com/microsoft/ai-agents-for-beginners>

What are AI Agents?

Large Language Models — They can understand natural language, reason about context, and turn a vague user request into a concrete plan of action.

Perform Actions — Without an agent system, an LLM just generates text. Inside an agent system, the LLM can *execute* steps — searching a database, calling an API, sending a message.

Access to Tools — What tools the agent can use depends on (1) the environment it's running in and (2) what the developer chose to give it. A travel agent might be able to search flights but not edit customer

Memory + Knowledge — Agents can have short-term memory (the current conversation) and long-term memory (a customer database, past interactions). The travel agent might "remember" that you prefer window seats.

Course Protocol

- Punctuality:
 - Arrive on time.
- Attendance Responsibility:
 - If you miss a class, it is your responsibility to catch up on any announcements, changes to deadlines, or materials covered.
- Device Etiquette:
 - Turn off or mute your cell phone during class.
- Classroom Conduct:
 - Refrain from engaging in activities that are irrelevant or distracting to others. No food is allowed.
- Active Participation:
 - Stay engaged and contribute to discussions and activities to enhance the learning experience for everyone.

AI tools

- Prompting LLMs such as ChatGPT is permitted for low-level programming questions or high-level conceptual questions about language models but **using it directly to solve the problem is prohibited.**
- **AI-generated solutions (for example, Assignments, Project report, Project presentation, Lab presentation) will not be accepted and will be considered cheating.**
- If AI use is necessary for any assignment, It will be announced when and how it can be used.

By 6pm, Sept 9th

Lab1: Image classification

1. Build a model in PyTorch
2. Train the model
3. Convert to ONNX
4. Quantize/compile ONNX to MXQ
5. Deploy MXQ on NPU



Input: x
Image pixels



Output: y
Object in image

Sponsor: Mobilint

- We would like to thank mobilint¹ for sponsoring for this class



MLA100 Low Profile
NPU PCIe Card

ARIES Specifications

Performance	Host Interface	Memory Capacity
80 TOPS	PCIe Gen 4.0 8-lane	16 GB (Optional 32 GB) LPDDR4, 4X
Memory Bandwidth	Power (TDP)	AI Performance
66.7 GB/s	25 W	MobileNetV2: 11,551 FPS
AI Performance	AI Performance	AI Performance
ResNet-50: 3,082 FPS	YOLOv9-S: 627 FPS	YOLOv9-C: 254 FPS

¹<https://www.mobilint.com/>

Group project (20%)

- Groups of 4 - 5 students will be formed to work on a semester-long project.
- The project should be based on the course materials covered in class. Some points would be deducted for any content that is not covered in class.
- Each group member is expected to participate in every phase of the project.
- Class engagement bonus might be applied for Group project presentation (TBD).
- The project will be graded based on the following components:
 - a) progress check-up: 5 pts
 - b) final presentation: 10 pts
 - c) technical report: 5 pts

Group project schedule

tentative and subject to change

- Aug 26: Formation of a group (on canvas)
- Sept 9: Project proposal (within 5 mins)
- Nov 4: Project progress checkup (within 7 mins) – 5 pts
- Dec 2: Submit presentation slides/ Project presentation (within 15 mins) – 10 pts
- Dec 2: Submit source codes and report (within 6 pages)– 5 pts

Group Project idea¹

NPU typically has a limited hardware resource such as memory and computing power.

Labs are designed to perform [256x256x3] pixel to [256x256x3] pixel.

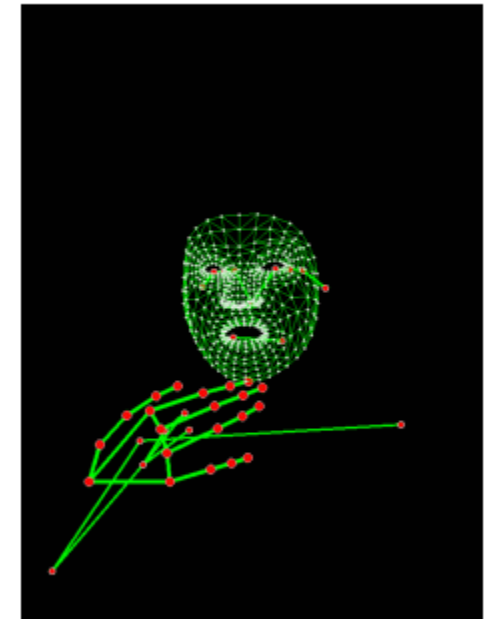
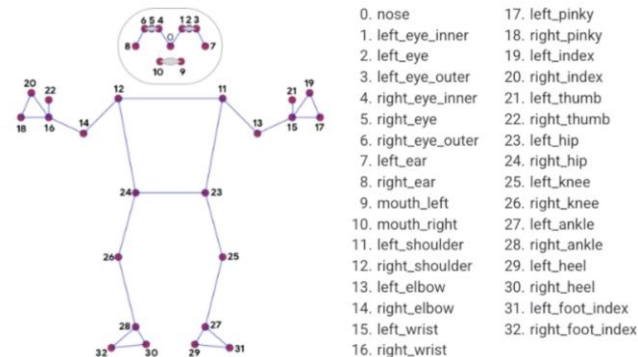
Instead, your task is to reconstruct sharp and clear images from degraded or blur or noised inputs and **upscale** it on NPU.

- Input: [256x256x3]
- Output: [1024x1024x3]
- On *Mobilint* NPU

Group Project idea²

Extract Key points from video

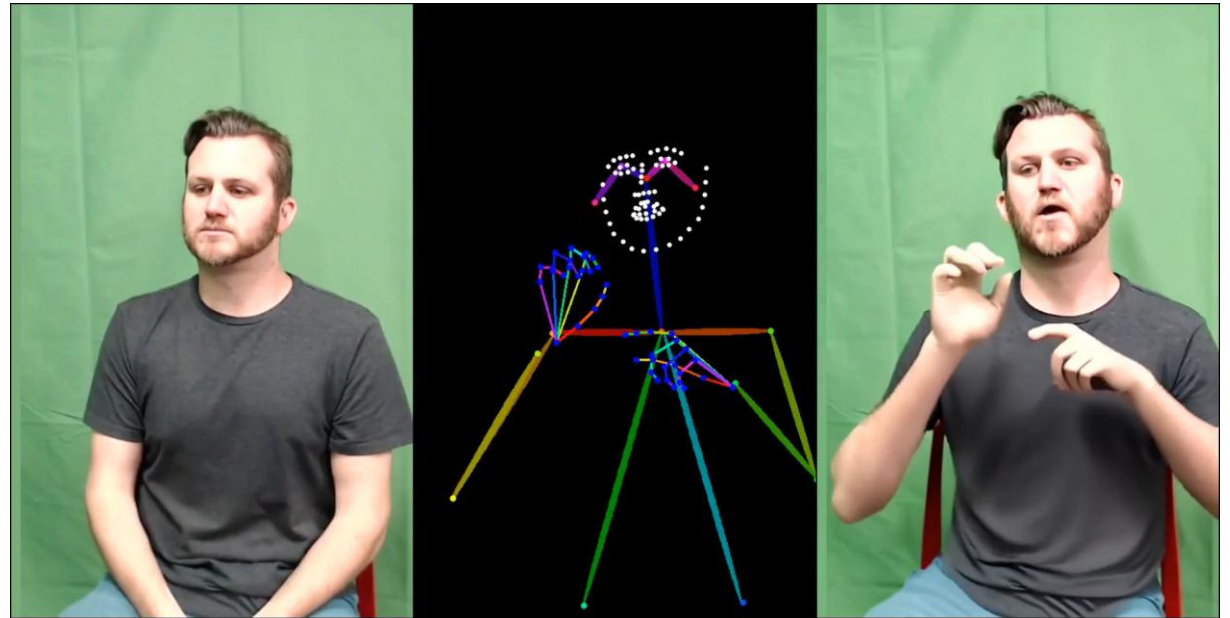
- Mouth movement
- Hand gestures
- Instead of relying on any package, build it from scratch



Group Project idea³

Generate video (movie clip)

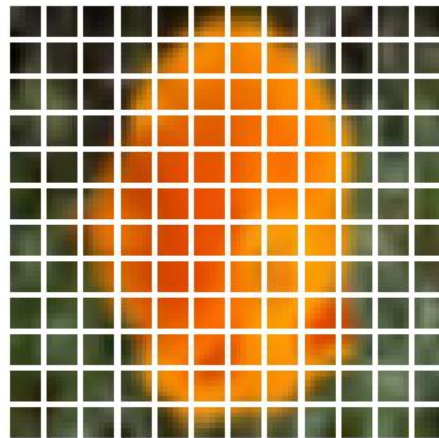
- Given an image and key points
- Generate signing video
- Instead of relying on any package, build it from scratch



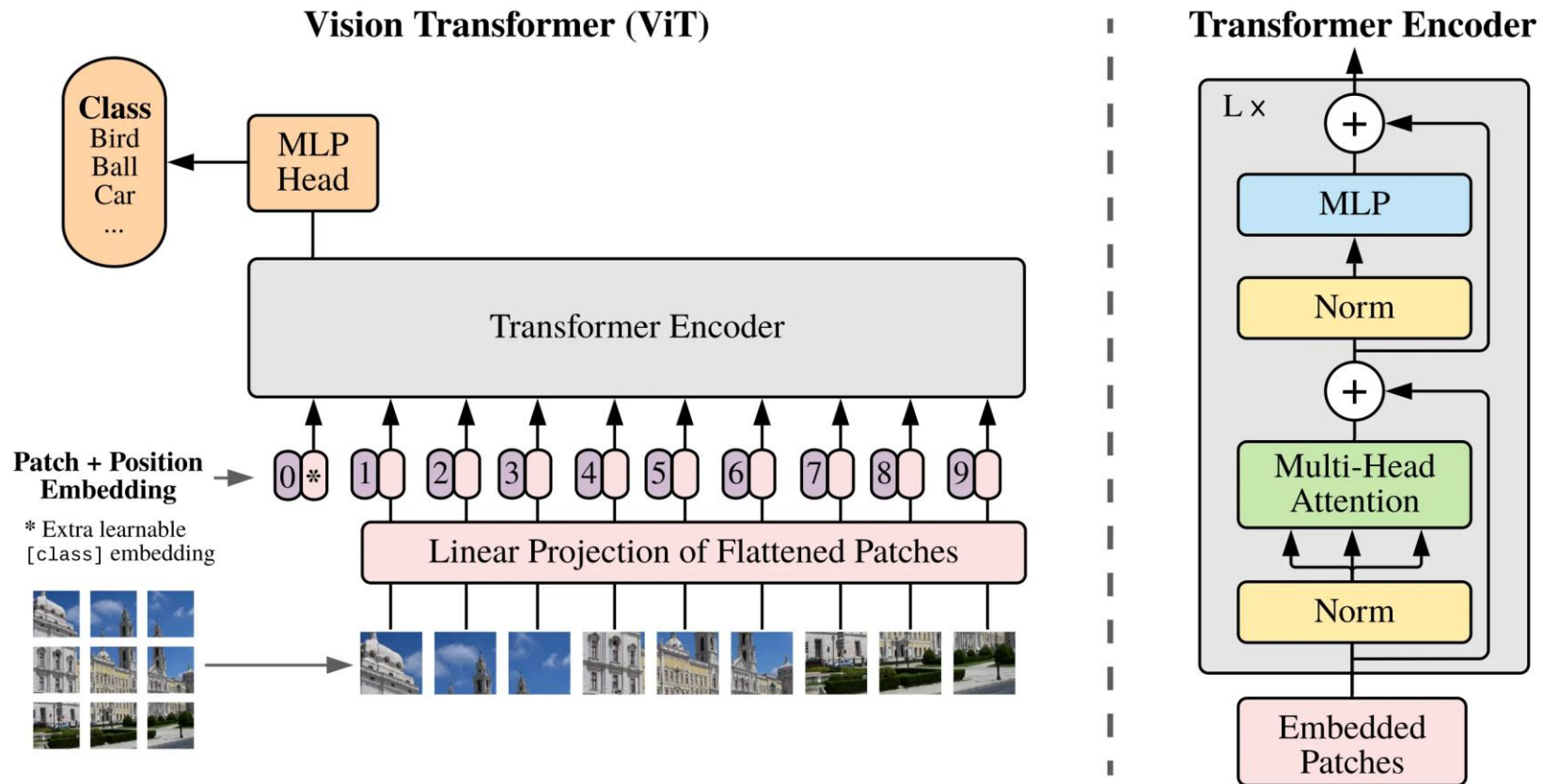
Group Project idea⁴

Image classification with Vision Transformer

- Instead of relying on any model, build it using PyTorch from scratch



Group Project idea⁴



Group Project idea⁴

Architecture of Vision Transformer

1. Patch Embedding
2. Position Embedding
3. Class Token
4. Transformer Encoder
5. MLP Head

Group Project idea⁵

YOLO (Object detection)

- Instead of relying on any model, build it using PyTorch from scratch

Era / Generation	Key Features & Modules	Structural Shift
Early Era (v1 - v3)	GoogleNet-based, Darknet-53 backbone, Feature Pyramid Networks (FPN)	Introduced the initial anchor-based, single-stage pipeline.
Mid Era (v4 - v7)	CSPDarknet, SPPF layer, PANet neck fusion	Maximized inference speeds through specialized feature-pooling and cross-stage paths.
Modern Era (v8 - v12)	C2f blocks, Decoupled Heads, Attention mechanisms	Shifted to Anchor-Free design with independent classification and regression branches.
Next-Gen (YOLO26+)	C3k2, SPPF residuals, NMS-free inference	Native end-to-end detection that removes post-processing logic for strict edge efficiency.

Group Project idea⁶

Agentic workflow

Build a function for object detection

Integrate with LLM (agentic workflow)

Apply a use-case, for example, counting crops or human in an image or video

Group Project idea⁷

Diffusion model

- Instead of relying on any model, build it using PyTorch from scratch

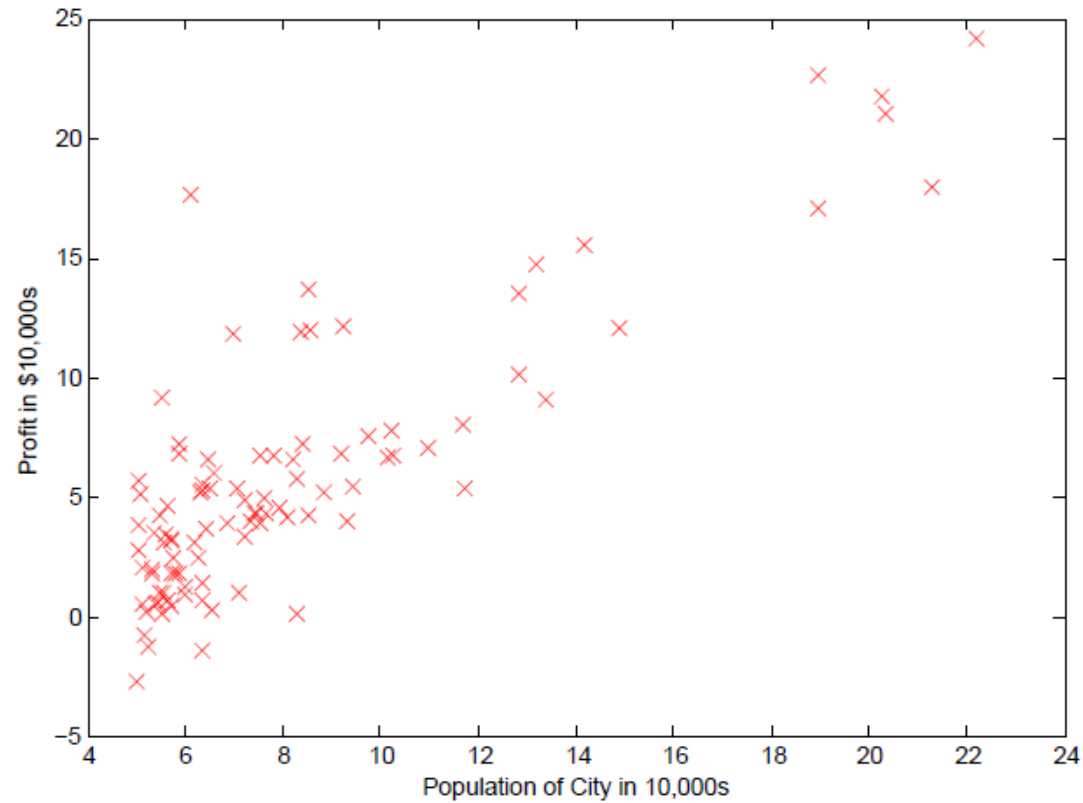
Core Components

- Forward Process (Diffusion)
- Reverse Process (Denoising)
- Timestep Conditioning
- Conditioning Guidance (Text embedding)

Linear regression

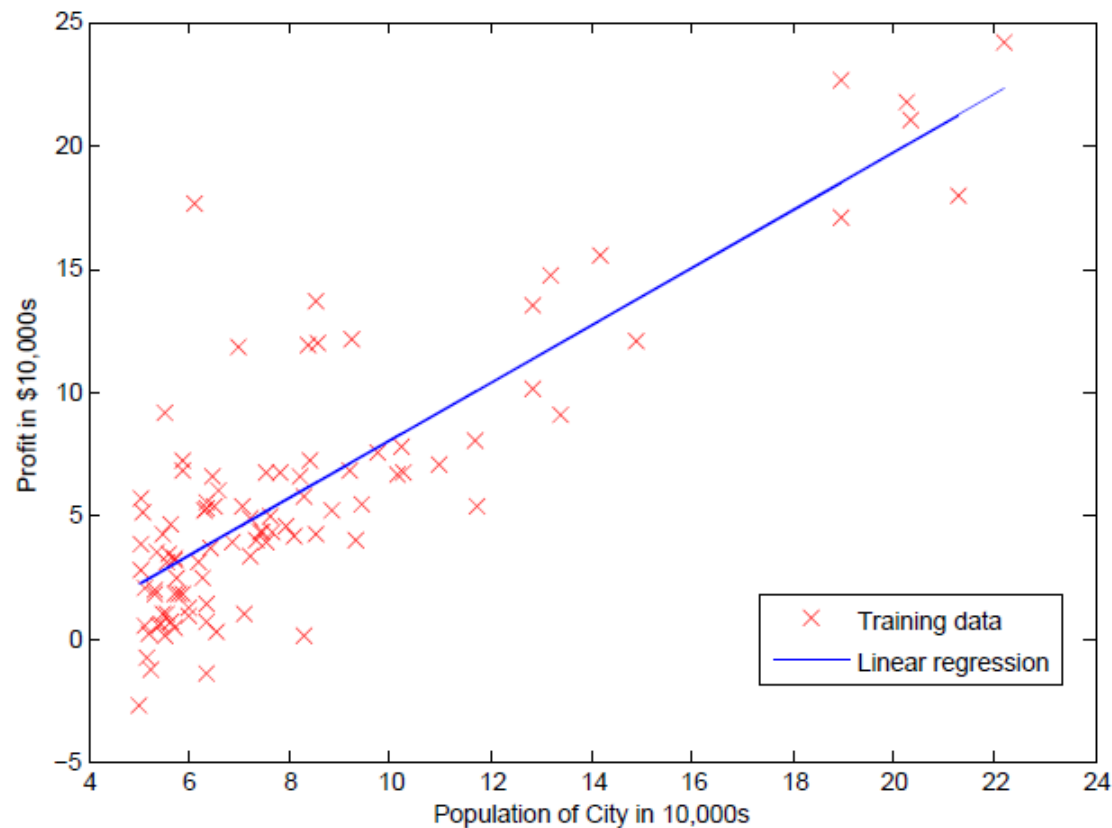
- Supervised learning
- Regression: predict continuous numerical values

Linear regression



Source: *Machine Learning*, Andrew Ng

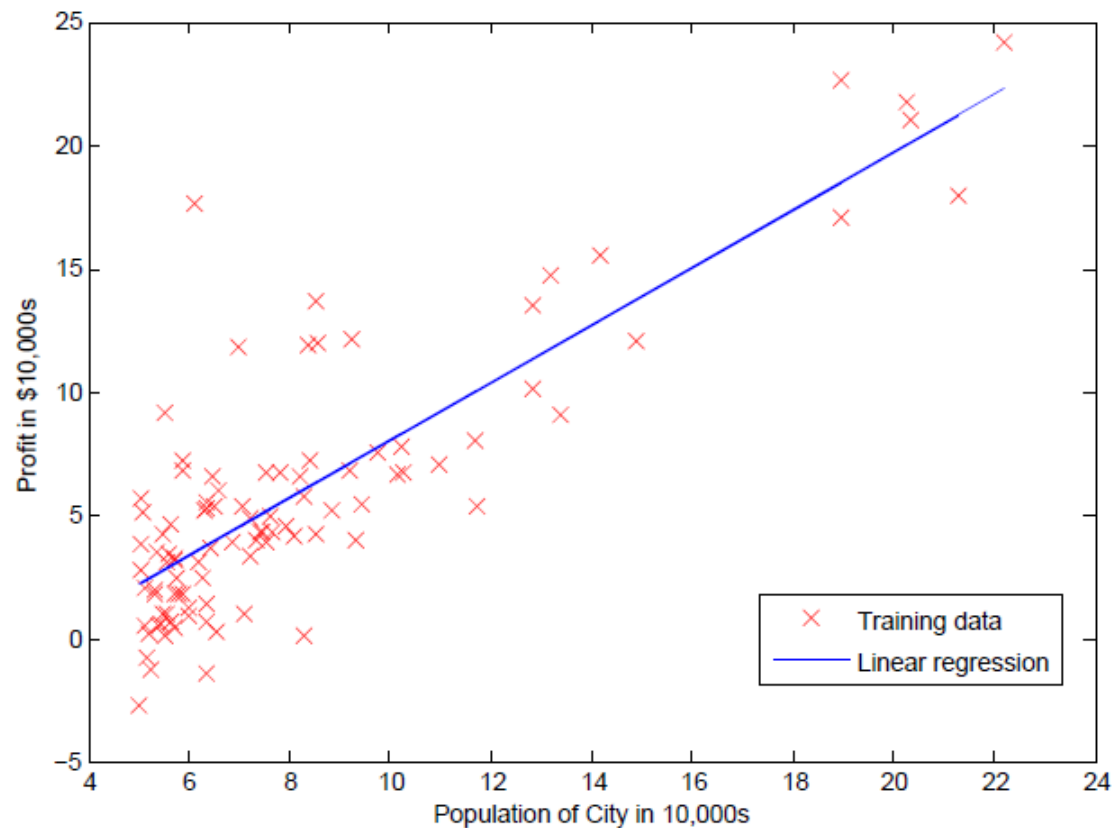
Linear regression



$$y = a \cdot x + b$$

- *a: slope*
- *b: bias*

Linear regression

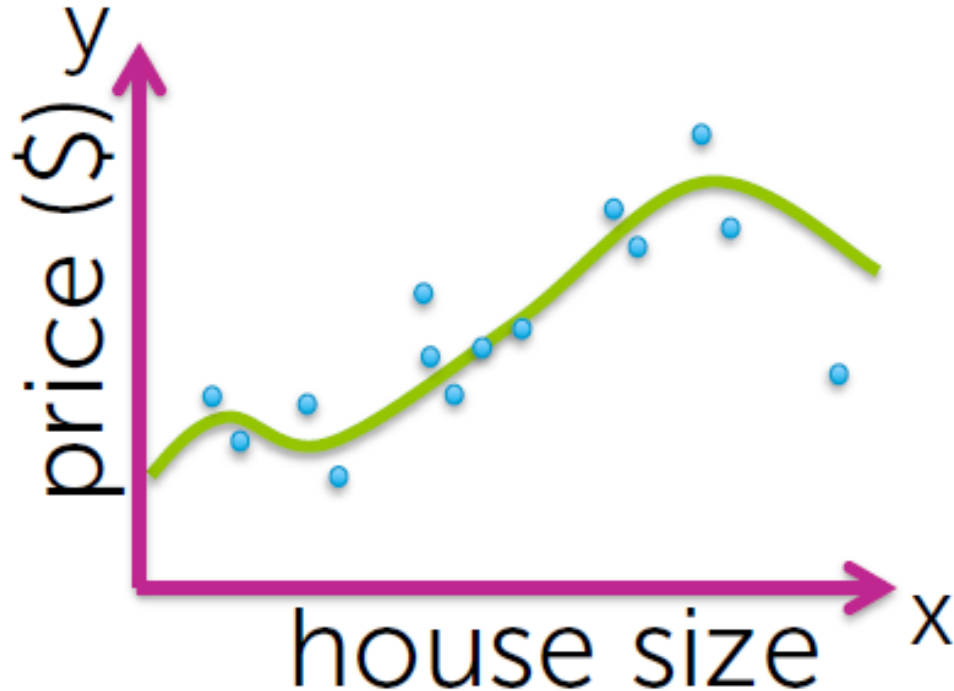


$$y = a \cdot x + b$$

$$\hat{y} = W_0 + W_1 \cdot x$$

- W_0, W_1 : Weights

Multiple variables Linear regression



More complex relationships than just a line.

$$\hat{y} = W_0 + W_1 \cdot x_1 + W_2 \cdot x_2 + \dots + W_n \cdot x_n$$

Binary classification

- The output of classification is a categorical value instead of a continuous numerical value.
- The output of binary classification is 1 or 0.
- For example,
 - Email: Spam / Not spam,
 - Online transaction: Fraudulent (Yes / No)

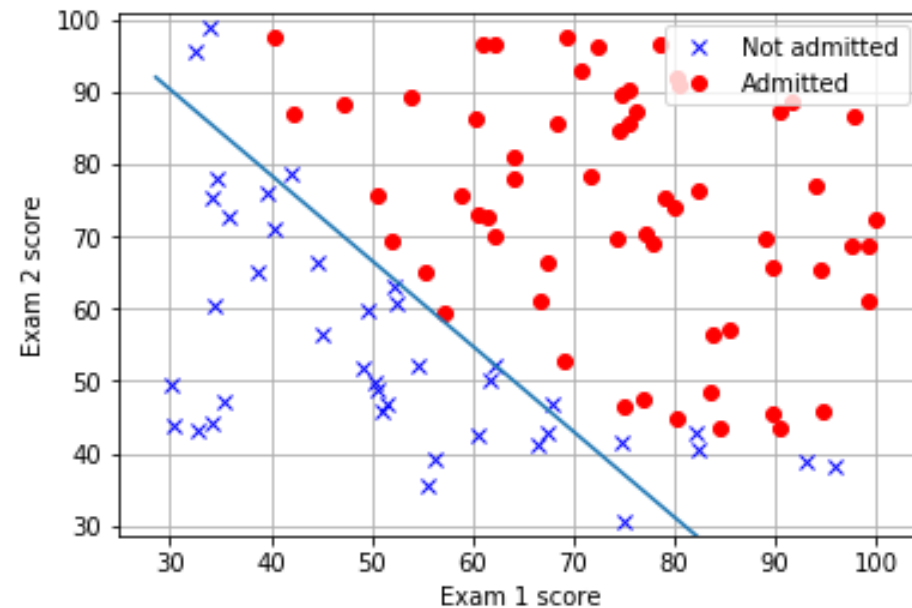
Logistic Regression

- Logistic regression is commonly used for binary classification.

Given x , the probability if $y = 1$

$$P(y = 1|x)$$

$$\hat{y} = \begin{cases} 1 & \text{if } P \geq 0.5 \\ 0 & \text{if } P < 0.5 \end{cases}$$



Logistic Regression

Given x , the probability if $y = 1$

$$\hat{y} = P(y = 1|x)$$

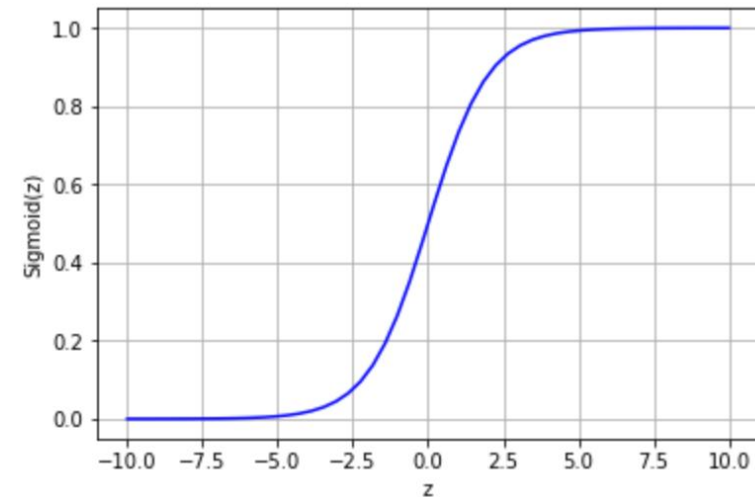
$$P = \sigma(z)$$

$$z = W_0 + W_1 \cdot x_1 + W_2 \cdot x_2 + \dots$$

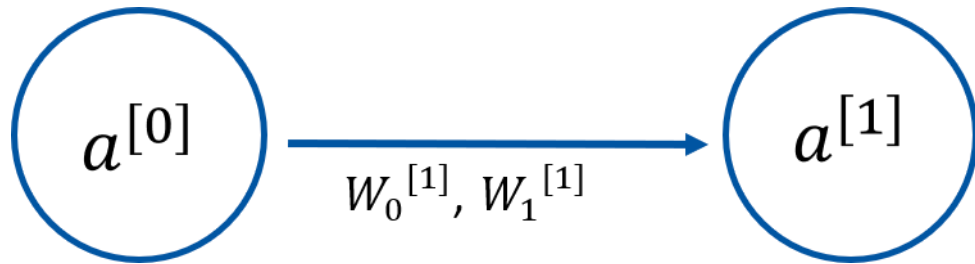
$$P = \frac{1}{1 + \exp(-(W_0 + W_1 \cdot x_1 + W_2 \cdot x_2))}$$

Sigmoid function

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$



Univariate logistic regression: Two neurons



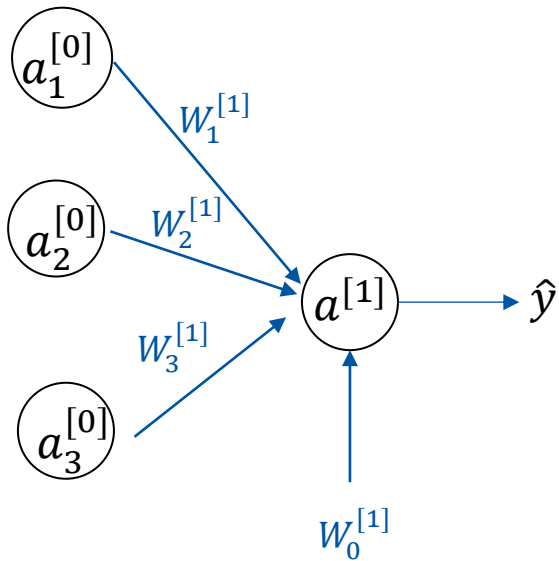
Input neuron: $x \rightarrow a^{[0]}$

$$Z^{[1]} = W_0^{[1]} + W_1^{[1]} \cdot a^{[0]}$$

Output neuron: $a^{[1]} = \sigma(Z^{[1]})$

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

Multivariate Logistic Regression

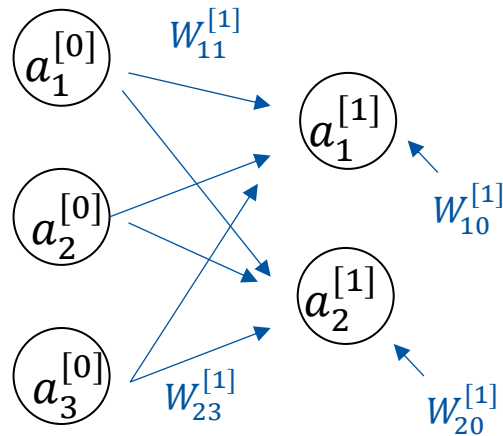


$$Z^{[1]} = W_0^{[1]} \cdot 1 + W_1^{[1]} \cdot a_1^{[0]} + W_2^{[1]} \cdot a_2^{[0]} + W_3^{[1]} \cdot a_3^{[0]}$$

$$a^{[1]} = \sigma(Z^{[1]})$$

$$\hat{y} = \begin{cases} 1 & \text{if } a^{[1]} \geq 0.5 \\ 0 & \text{if } a^{[1]} < 0.5 \end{cases}$$

Multiclass classifier



Input layer

Output layer

$$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} \cdot a_1^{[0]} + W_{12}^{[1]} \cdot a_2^{[0]} + W_{13}^{[1]} \cdot a_3^{[0]}$$

$$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} \cdot a_1^{[0]} + W_{22}^{[1]} \cdot a_2^{[0]} + W_{23}^{[1]} \cdot a_3^{[0]}$$

$$a_1^{[1]} = \sigma(Z_1^{[1]})$$

$$a_2^{[1]} = \sigma(Z_2^{[1]})$$

Hidden Layer

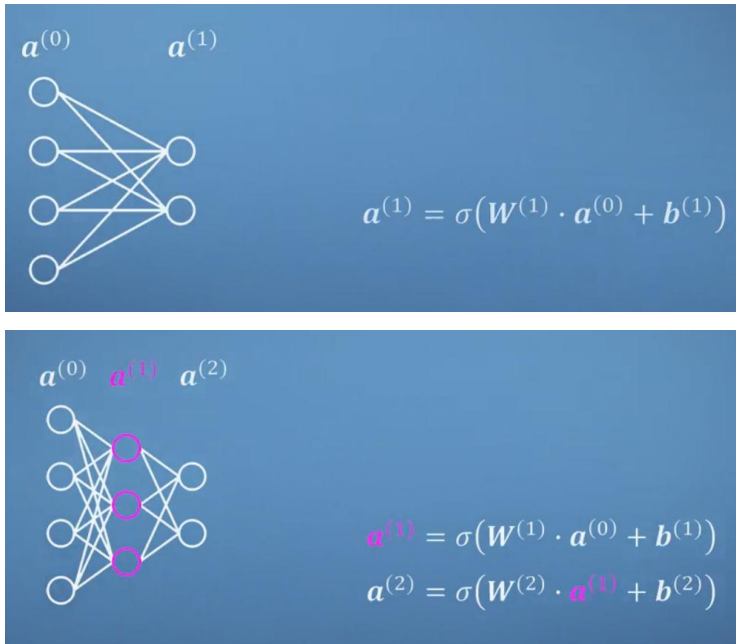


image: Mathematics for Machine Learning

- Another layer (hidden layer) can be inserted between input layer and output layer.
- It works well if there is no direct (or strong) correlation between input layer and output layer.

Neural network

- Machine Learning : No Hidden layer
- (Shallow) Neural Network : One hidden layer
- Deep Neural network: More than two hidden layers

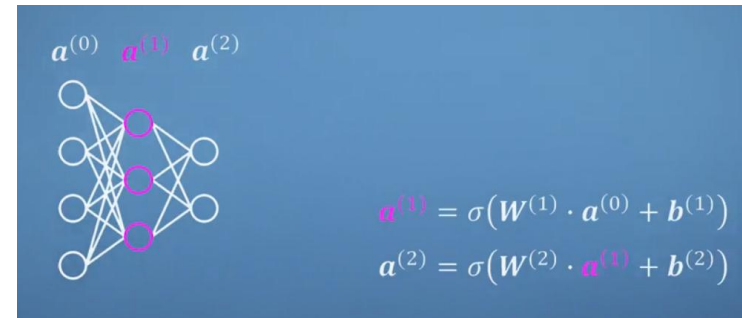
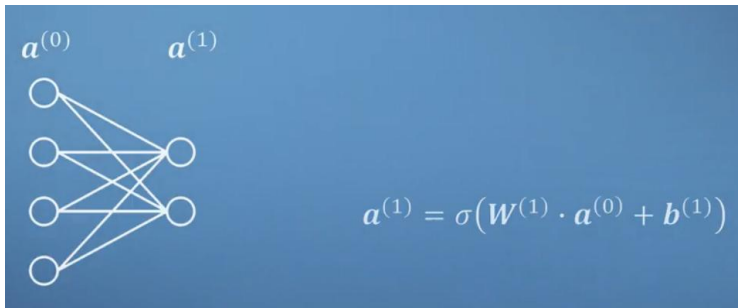
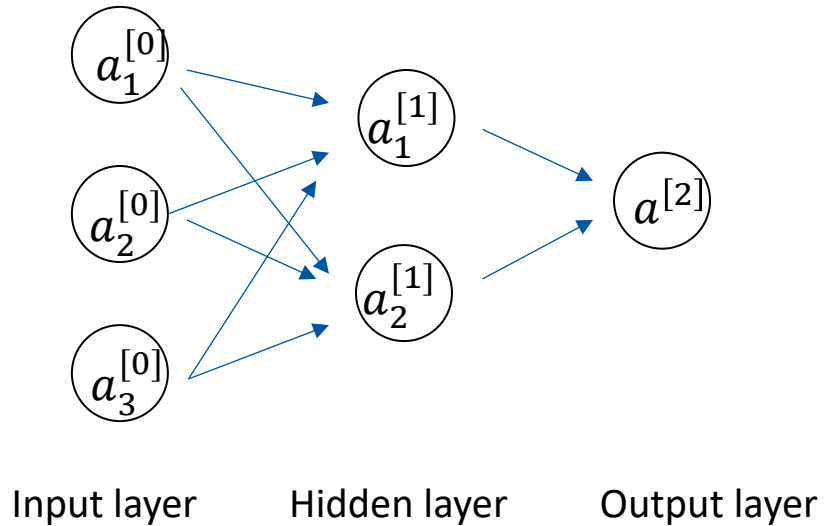


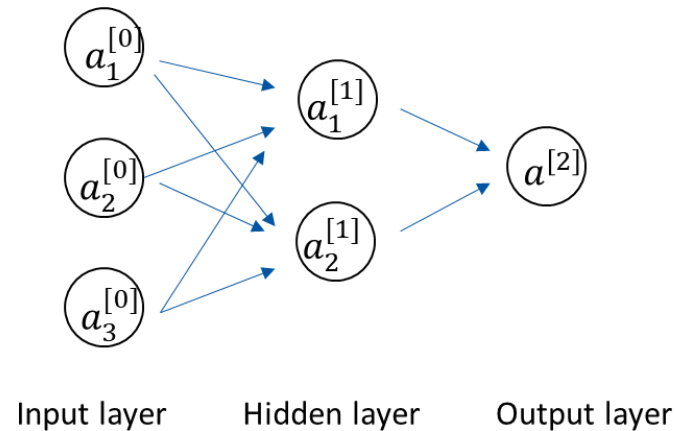
image: Mathematics for Machine Learning

Shallow Neural Network



- Input layer : 3 neurons
- Hidden layer : 2 neurons
- output layer : 1 neurons

Shallow Neural Network



- Input layer : 3 neurons
- Hidden layer : 2 neurons
- output layer : 1 neurons

Hidden layer

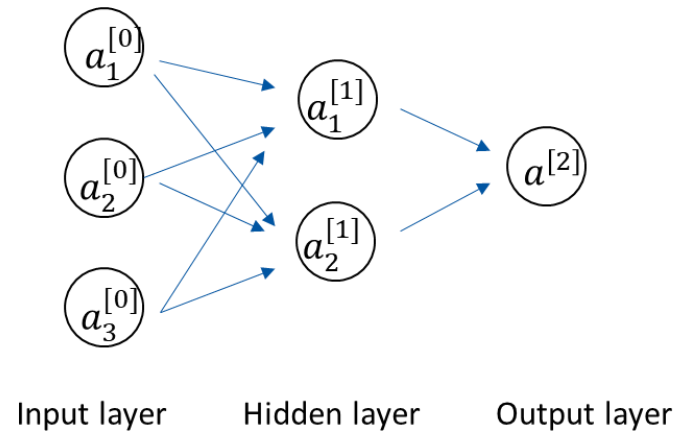
$$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} \cdot a_1^{[0]} + W_{12}^{[1]} \cdot a_2^{[0]} + W_{13}^{[1]} \cdot a_3^{[0]}$$

$$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} \cdot a_1^{[0]} + W_{22}^{[1]} \cdot a_2^{[0]} + W_{23}^{[1]} \cdot a_3^{[0]}$$

$$a_1^{[1]} = \sigma(Z_1^{[1]})$$

$$a_2^{[1]} = \sigma(Z_2^{[1]})$$

Shallow Neural Network



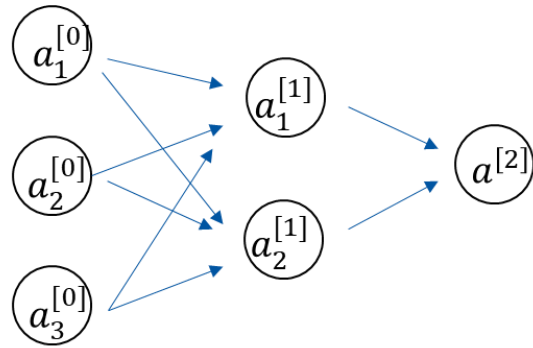
- Input layer : 3 neurons
- Hidden layer : 2 neurons
- output layer : 1 neurons

Output layer

$$Z^{[2]} = W_{10}^{[2]} + W_{11}^{[2]} \cdot a_1^{[1]} + W_{12}^{[2]} \cdot a_2^{[1]}$$

$$a^{[2]} = \sigma(Z^{[2]})$$

Neural network with matrix form



Input layer Hidden layer Output layer

- Input layer : 3 neurons
- Hidden layer : 2 neurons
- output layer : 1 neurons

Hidden layer

$$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} \cdot a_1^{[0]} + W_{12}^{[1]} \cdot a_2^{[0]} + W_{13}^{[1]} \cdot a_3^{[0]}$$

$$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} \cdot a_1^{[0]} + W_{22}^{[1]} \cdot a_2^{[0]} + W_{23}^{[1]} \cdot a_3^{[0]}$$

$$a_1^{[1]} = \sigma(Z_1^{[1]})$$

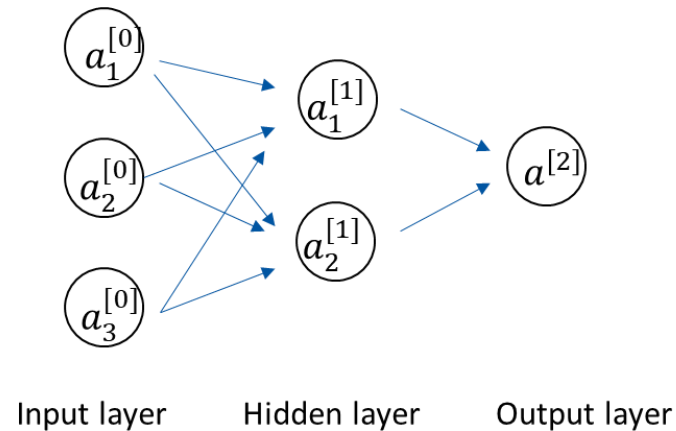
$$a_2^{[1]} = \sigma(Z_2^{[1]})$$

$$W^{[1]} : [2,4]$$

$$W^{[1]} : [n_h, n_x+1]$$

(hidden-node, input-node)

Neural network: matrix dimension



- Input layer : 3 neurons
- Hidden layer : 2 neurons
- output layer : 1 neurons

Output layer

$$Z^{[2]} = W_{10}^{[2]} + W_{11}^{[2]} \cdot a_1^{[1]} + W_{12}^{[2]} \cdot a_2^{[1]}$$

$$a^{[2]} = \sigma(Z^{[2]})$$

$$W^{[2]} : [1,3]$$

$$W^{[2]} : [n_o, n_h+1]$$

(output-node, hidden-node)

Initialize weights

Weights should be initialized randomly rather than to all zeros.

If weights are initialized to be zero, then, each neuron in the first hidden layer will perform the same computation. So even after multiple iterations of gradient descent, each neuron in the layer will be computing the same thing as other neurons.

Weights initialization

Need to break symmetry.

If $W^{[1]}$, $W_0^{[1]}$, $W^{[2]}$, $W_0^{[2]}$ are started with zero,

Then, $Z^{[1]}$, $Z^{[2]}$ become zero.

$a^{[1]}$, $a^{[2]}$ become 0.5.

$dW^{[2]}$ is same

$a^{[1]}$, $a^{[2]}$ will always get same updated!

Small random numbers

$W = 0.01 * \text{np.random.randn}(D, H)$

samples from a zero mean, unit standard deviation Gaussian

Randomness

Randomness shows up in many places: parameter initialization, dropout, data ordering, etc.

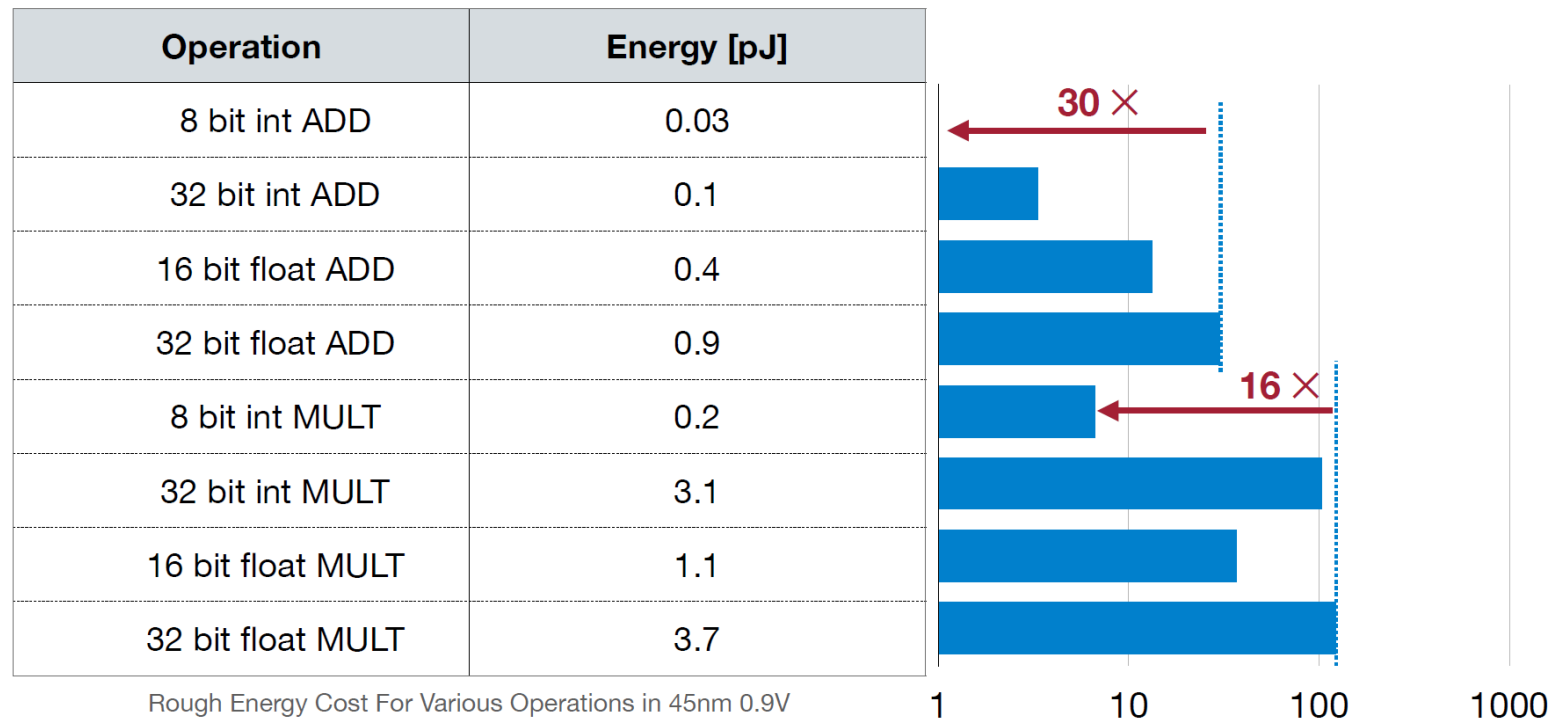
For reproducibility, we recommend you always pass in a different random seed for each use of randomness.

Determinism is particularly useful when debugging, so you can hunt down the bug.

```
# set random seed  
seed = 0  
torch.manual_seed(seed)
```

Low Bit Operations are Cheaper

Less Bit-Width → Less Energy



Integer

- Unsigned Integer
 - n -bit Range: $[0, 2^n - 1]$
- Signed Integer
 - n -bit Range: $[-2^{n-1}, 2^{n-1} - 1]$

0	0	1	1	0	0	0	1
x	x	x	x	x	x	x	x

$$2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 = 49$$

Sign Bit

1	0	1	1	0	0	0	1
	x	x	x	x	x	x	x

$$- 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 = -49$$

Fixed-Point Number



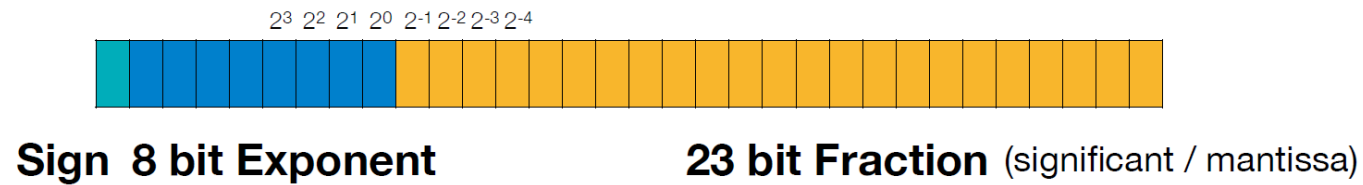
Integer . Fraction

“Decimal” Point



$$\begin{array}{cccccccc} \times & \times & \times & \times & \times & \times & \times & \times \\ -2^3 & +2^2 & +2^1 & +2^0 & +2^{-1} & +2^{-2} & +2^{-3} & +2^{-4} \end{array} = 3.0625$$

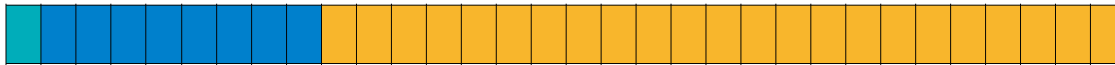
Floating-Point Number



Floating-Point Number

Exponent Width → Range; Fraction Width → Precision

[IEEE 754](#) Single Precision 32-bit Float (IEEE FP32)



[IEEE 754](#) Half Precision 16-bit Float (IEEE FP16)



[Google](#) Brain Float (BF16)



Exponent (bits)	Fraction (bits)	Total (bits)
8	23	32
5	10	16
8	7	16

Floating-Point Number

Exponent Width → Range; Fraction Width → Precision

[IEEE 754](#) Single Precision 32-bit Float (IEEE FP32)



Exponent
(bits)

8

Fraction
(bits)

23

Total
(bits)

32

[IEEE 754](#) Half Precision 16-bit Float (IEEE FP16)



5

10

16

[Nvidia](#) FP8 (E4M3)



* FP8 E4M3 does not have INF, and $S.1111.111_2$ is used for NaN.
* Largest FP8 E4M3 normal value is $S.1111.110_2 = 448$.

4

3

8

[Nvidia](#) FP8 (E5M2) for gradient in the backward



* FP8 E5M2 have INF ($S.11111.00_2$) and NaN ($S.11111.XX_2$).
* Largest FP8 E5M2 normal value is $S.11110.11_2 = 57344$.

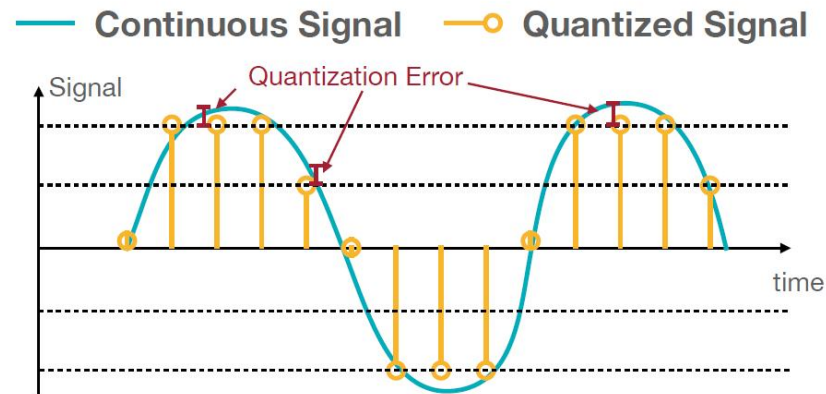
5

2

8

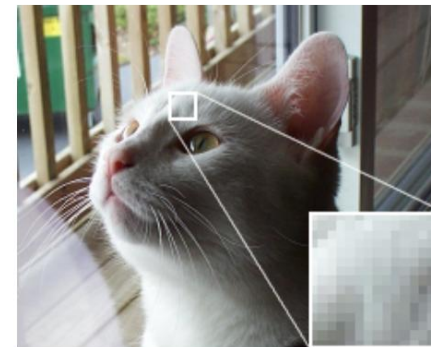
Quantization

Quantization is the process of constraining an input from a continuous or otherwise large set of values to a discrete set

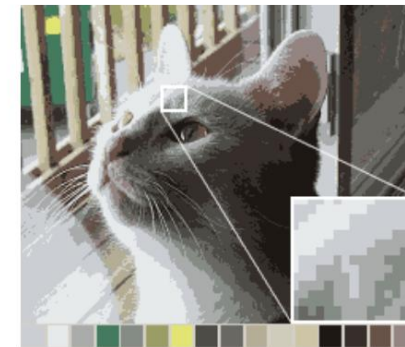


The difference between an input value and its quantized value is referred to as quantization error.

Original Image



16-Color Image



Images are in the public domain.

“Palettization”

Quantization

quantization is a linear mapping from floating point values to quantized integer values

